$$Nmax = \sqrt{\frac{(1-\alpha)}{\beta}}$$

The resulting capacity gain is relative to the minimum throughput, so the absolute system capacity is obtained as follows:

$$Xmax = X\,(1) \times C\,(Nmax)$$

1.2 Database connections boundaries

Every connection requires a TCP socket from the client (application) to the server (database).

The total number of connections offered by a database server depends on the underlying hardware resources, and finding how many connections a server can handle is possible through measurements and proven scalability models.

> SQL Server 2016[a] and MySQL 5.7[b] use thread-based connection handling.
>
> PostgreSQL 9.5[c] uses one operating system process for each individual connection.
>
> On Windows systems, Oracle uses threads, while on Linux, it uses process-based connections. Oracle 12c[d] comes with a thread-based connection model for Linux systems too.
>
> [a] https://msdn.microsoft.com/en-us/library/ms190219.aspx
> [b] https://dev.mysql.com/doc/refman/5.7/en/connection-threads.html
> [c] http://www.postgresql.org/docs/current/static/connect-estab.html
> [d] http://docs.oracle.com/database/121/CNCPT/process.htm

A look into database system internals reveals the tight dependency on CPU, Memory, and Disk resources. Because I/O operations are costly, the database uses a buffer pool to map into memory the underlying data and index pages. Changes are first applied in memory and flushed to disk in batches to achieve better write performance.

Even if all indexes are entirely cached in memory, disk access might still occur if the requested data blocks are not cached into the memory buffer pool. Not just queries may generate I/O traffic, but the transaction and the redo logs require flushing in-memory data structures periodically so that durability is not compromised.

To provide data integrity, any relational database system must use exclusive locks to protect data blocks (rows and indexes) from being updated concurrently. This is true even if the database system uses MVCC (Multi-Version Concurrency Control) because otherwise atomicity would be compromised. This topic is going to be discussed in greater detail in the Transactions chapter.

This means that high-throughput database applications experience contention on CPU, Memory, Disk, and Locks. When all the database server resources are in use, adding more workload only increases contention, therefore lowering throughput.

Resources might get saturated due to improper system configuration, so the first step to improving a system throughput is to tune it according to the current data access patterns.

Lowering response time not only makes the application more responsive, but it can also increase throughput.

However, response time alone is not sufficient in a highly concurrent environment. To maintain a fixed upper bound response time, the system capacity must increase relative to the incoming request throughput. Adding more resources can improve scalability up to a certain point, beyond which the capacity gain starts dropping.

At the Velocity conference[a], both Google Search and Microsoft Bing teams have concluded that higher response times can escalate and even impact the business metrics.

Capacity planning is a feedback-driven mechanism, and it requires constant application monitoring, and so, any optimization must be reinforced by application performance metrics.

[a] http://radar.oreilly.com/2009/06/bing-and-google-agree-slow-pag.html

1.3 Scaling up and scaling out

Scaling is the effect of increasing capacity by adding more resources. Scaling vertically (scaling up) means adding resources to a single machine. Increasing the number of available machines is called horizontal scaling (scaling out).

Traditionally, adding more hardware resources to a database server has been the preferred way of increasing database capacity. Relational databases have emerged in the late seventies, and, for two and a half decades, the database vendors took advantage of the hardware advancements following the trends in Moore's Law.

Distributed systems are much more complex to manage than centralized ones, and that is why horizontal scaling is more challenging than scaling vertically. On the other hand, for the same price of a dedicated high-performance server, one could buy multiple commodity machines whose sum of available resources (CPU, Memory, Disk Storage) is greater than of the single dedicated server. When deciding which scaling method is better suited for a given enterprise system, one must take into account both the price (hardware and licenses) and the inherent developing and operational costs.

Being built on top of many open source projects (e.g. PHP, MySQL), Facebook[3] uses a horizontal scaling architecture to accommodate its massive amounts of traffic.

StackOverflow[4] is the best example of a vertical scaling architecture. In one of his blog posts[5], Jeff Atwood explained that the price of Windows and SQL Server licenses was one of the reasons for not choosing a horizontal scaling approach.

No matter how powerful it might be, one dedicated server is still a single point of failure, and throughput drops to zero if the system is no longer available. For this reason, database replication is not optional in many enterprise systems.

1.3.1 Master-Slave replication

For enterprise systems where the read/write ratio is high, a Master-Slave replication scheme is suitable for increasing availability.

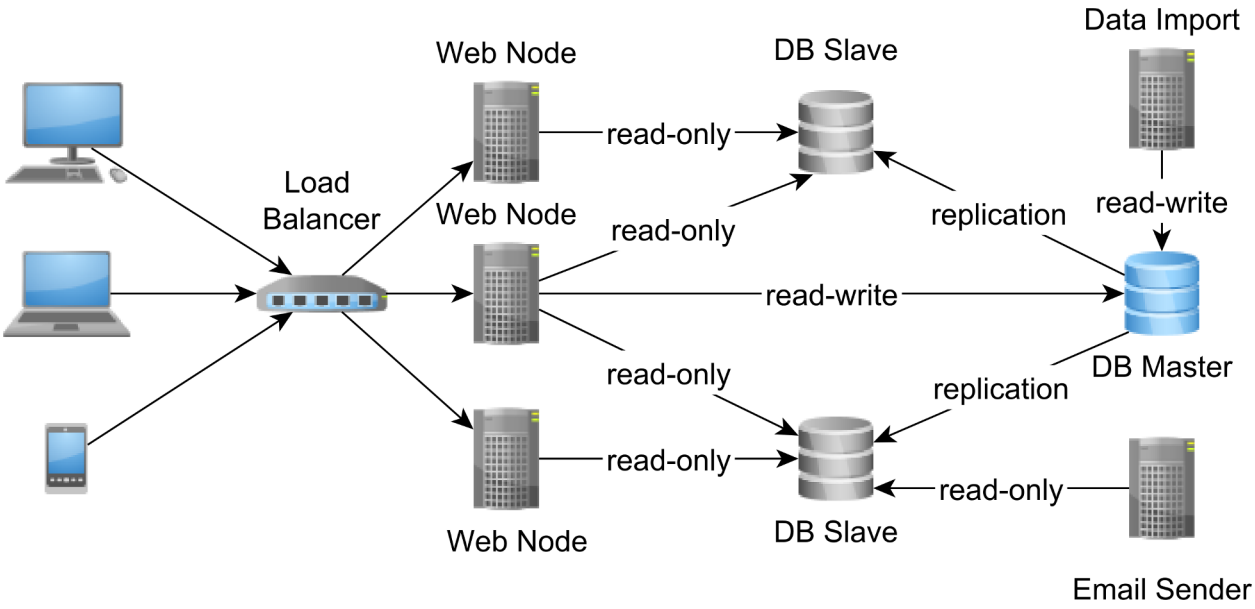

Figure 1.2: Master-Slave replication

The Master is the system of record and the only node accepting writes. All changes recorded by the Master node are replayed onto Slaves as well. A binary replication uses the Master node WAL (Write Ahead Log) while a statement-based replication replays on the Slave machines the exact statements executed on Master.

Asynchronous replication is very common, especially when there are more Slave nodes to update.

[3] https://www.facebook.com/note.php?note_id=409881258919
[4] http://stackexchange.com/performance
[5] http://blog.codinghorror.com/scaling-up-vs-scaling-out-hidden-costs/

The Slave nodes are eventual consistent as they might lag behind the Master. In case the Master node crashes, a cluster-wide voting process must elect the new Master (usually the node with the most recent update record) from the list of all available Slaves.

The asynchronous replication topology is also referred as *warm standby* because the election process does not happen instantaneously.

Most database systems allow one synchronous Slave node, at the price of increasing transaction response time (the Master has to block waiting for the synchronous Slave node to acknowledge the update). In case of Master node failure, the automatic failover mechanism can promote the synchronous Slave node to become the new Master.

Having one synchronous Slave allows the system to ensure data consistency in case of Master node failures since the synchronous Slave is an exact copy of the Master. The synchronous Master-Slave replication is also called a *hot standby* topology because the synchronous Slave is readily available for replacing the Master node.

When only asynchronous Slave nodes are available, the newly elected Slave node might lag behind the failed Master, in which case consistency and durability are traded for lower latencies and higher throughput.

Aside from eliminating the single point of failure, database replication can also increase transaction throughput without affecting response time. In a Master-Slave topology, the Slave nodes can accept read-only transactions, therefore routing read traffic away from the Master node.

The Slave nodes increase the available read-only connections and reduce Master node resource contention, which, in turn, can also lower read-write transaction response time. If the Master node can no longer keep up with the ever-increasing read-write traffic, a Multi-Master replication might be a better alternative.

1.3.2 Multi-Master replication

In a Multi-Master replication scheme, all nodes are equal and can accept both read-only and read-write transactions. Splitting the load among multiple nodes can only increase transaction throughput and reduce response time as well.

However, because distributed systems are all about trade-offs, ensuring data consistency is challenging in a Multi-Master replication scheme because there is no longer a single source of truth. The same data can be modified concurrently on separate nodes, so there is a possibility of conflicting updates. The replication scheme can either avoid conflicts or it can detect them and apply an automatic conflict resolution algorithm.

To avoid conflicts, the two-phase commit protocol can be used to enlist all participating nodes in one distributed transaction. This design allows all nodes to be in sync at all time, at the cost of increasing transaction response time (by slowing down write operations).

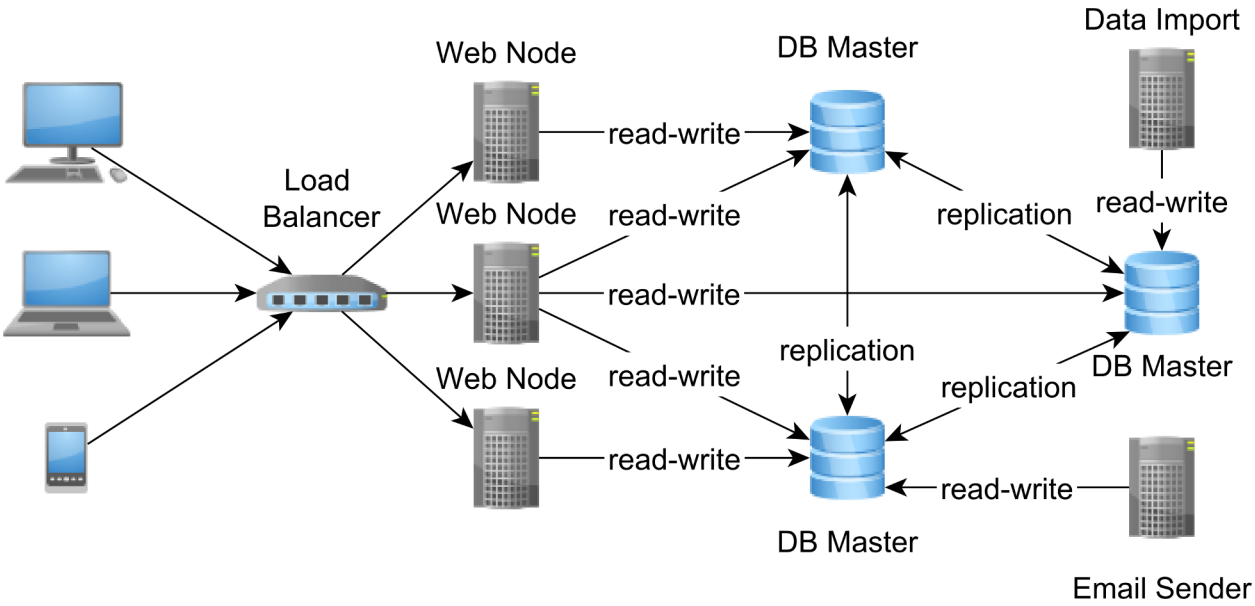

Figure 1.3: Multi-Master replication

If nodes are separated by a WAN (Wide Area Network), synchronization latencies may increase dramatically. If one node is no longer reachable, the synchronization will fail, and the transaction will roll back on all Masters.

Although avoiding conflicts is better from a data consistency perspective, synchronous replication might incur high transaction response times. On the other hand, at the price of having to resolve update conflicts, asynchronous replication can provide better throughput,

The asynchronous Multi-Master replication requires a conflict detection and an automatic conflict resolution algorithm. When a conflict is detected, the automatic resolution tries to merge the two conflicting branches, and, in case it fails, manual intervention is required.

1.3.3 Sharding

When data size grows beyond the overall capacity of a replicated multi-node environment, splitting data becomes unavoidable. Sharding means distributing data across multiple nodes, so each instance contains only a subset of the overall data.

Traditionally, relational databases have offered *horizontal partitioning* to distribute data across multiple tables within the same database server. As opposed to *horizontal partitioning*, sharding requires a distributed system topology so that data is spread across multiple machines.

Each shard must be self-contained because a user transaction can only use data from within a single shard. Joining across shards is usually prohibited because the cost of distributed locking and the networking overhead would lead to long transaction response times.

By reducing data size per node, indexes also require less space, and they can better fit into main memory. With less data to query, the transaction response time can also get shorter too.

The typical sharding topology includes, at least, two separate data centers.

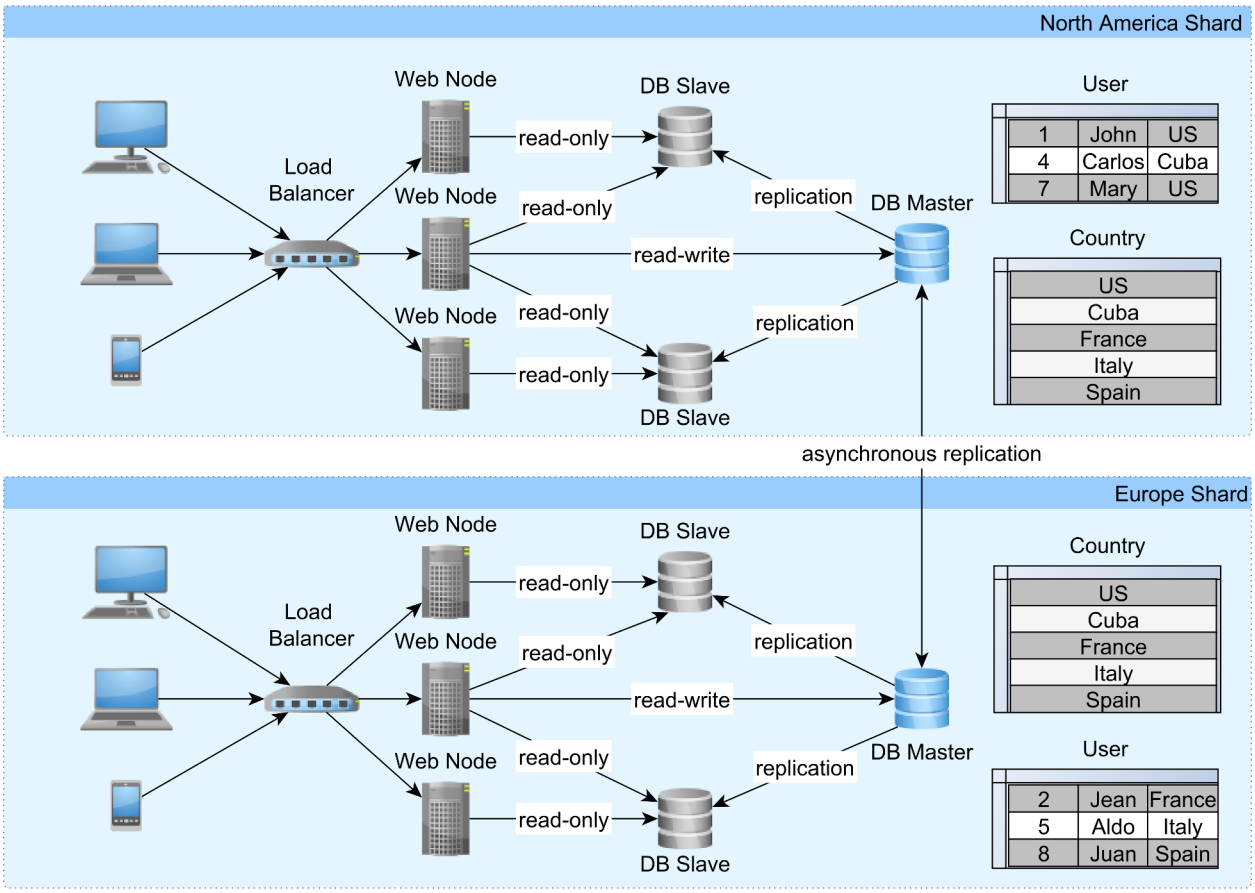

Figure 1.4: Sharding

Each data center can serve a dedicated geographical region, so the load is balanced across geographical areas. Not all tables need to be partitioned across shards, smaller size ones being duplicated on each partition. To keep the shards in sync, an asynchronous replication mechanism can be employed.

In the previous diagram, the country table is mirrored from one data center to the other, and partitioning happens on the user table only. To eliminate the need for inter-shard data processing, each user along with all user-related data are contained in one data center only.

In the quest for increasing system capacity, sharding is usually a last resort strategy, employed after exhausting all other available options, such as:

- optimizing the data layer to deliver lower transaction response times
- scaling each replicated node to a cost-effective configuration
- adding more replicated nodes until synchronization latencies start dropping below an acceptable threshold.

MySQL cluster auto-sharding

MySQL Cluster[a] offers automatic sharding, so data is evenly distributed (using a primary key hashing function) over multiple commodity hardware machines. Every node accepts both read and write transactions and, just like Multi-Master replication, conflicts are automatically discovered and resolved.

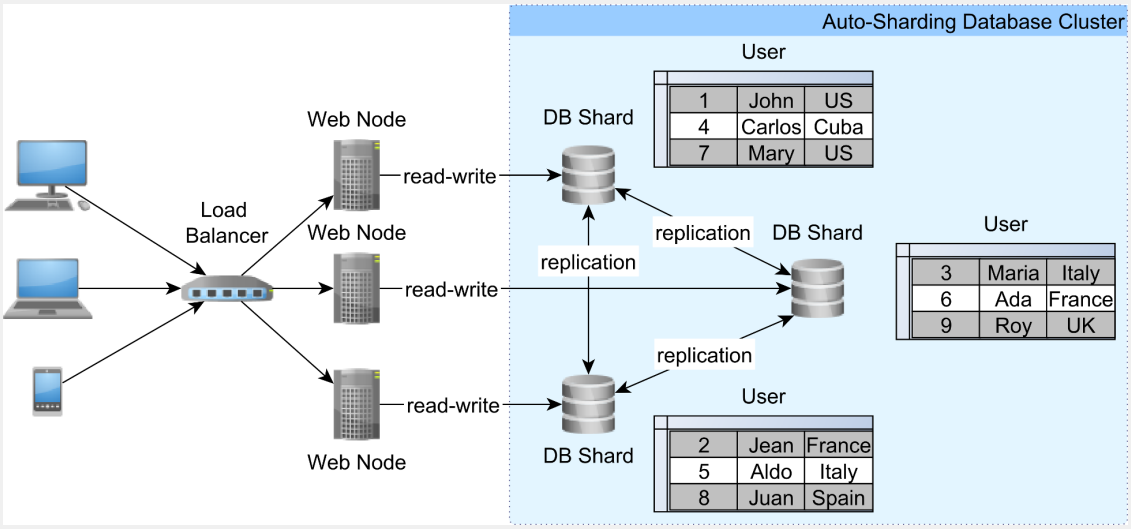

Figure 1.5: Auto-sharding

The auto-sharding topology is similar to the Multi-Master replication architecture as it can increase throughput by distributing incoming load to multiple machines. While in a Multi-Master replicated environment every node stores the whole database, the auto-sharding cluster distributes data so that each shard is only a subset of the whole database.

Because the cluster takes care of distributing data, the application does not have to provide a data shard routing layer, and SQL joins are possible even across different shards. MySQL Cluster 7.3 uses the *NDB* storage engine, and so it lacks some features provided by InnoDB[b] like multiple transaction isolation levels or MVCC (Multi-Version Concurrency Control).

[a] https://www.mysql.com/products/cluster/scalability.html
[b] http://dev.mysql.com/doc/mysql-cluster-excerpt/5.6/en/mysql-cluster-ndb-innodb-engines.html

Little's Law

In any given system, the ultimate relationship between response time and throughput is given by Little's Law[a] and, high values of incoming throughput can cause an exponential growth in response time due to resource saturation.

Nevertheless, when taking a single database connection, by lowering the average transaction response time, more transactions can be accommodated in a given time unit. For this reason, the following chapters explain in greater detail what is needed to be done in order to reduce transaction response time as much as possible.

[a] https://people.cs.umass.edu/~emery/classes/cmpsci691st/readings/OS/Littles-Law-50-Years-Later.pdf

2. JDBC Connection Management

The JDBC (Java Database Connectivity) API provides a common interface for communicating with a database server. All the networking logic and the database-specific communication protocol are hidden away behind the vendor-independent JDBC API. For this reason, all the JDBC interfaces must be implemented according to the database vendor-specific requirements. The `java.sql.Driver` is the main entry point for interacting with the JDBC API, defining the implementation version details and providing access to a database connection.

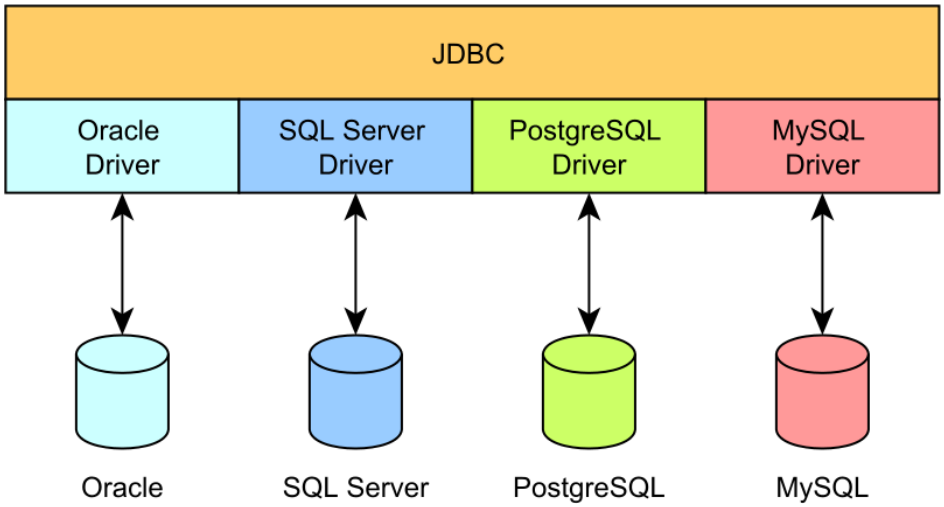

Figure 2.1: JDBC plugin architecture

JDBC defines four driver types:

- Type 1: It is only a bridge to an actual ODBC driver implementation.
- Type 2: It uses a database-specific native client implementation (e.g. *Oracle Call Interface*).
- Type 3: It delegates calls to an application server offering database connectivity support.
- Type 4: The JDBC driver implements the database communication protocol solely in Java.

Being easier to setup and debug, the Type 4 driver is usually the preferred alternative.

To communicate with a database server, a Java program must first obtain a `java.sql.Connection`. Although the `java.sql.Driver` is the actual database connection provider, it is more convenient to use the `java.sql.DriverManager` since it can also resolve the JDBC driver associated with the current database connection URL.

> Previously, the application required to load the driver prior to establishing a connection but, since JDBC 4.0, the *Service Provider Interfaces* mechanism can automatically discover all the available drivers in the current application classpath.

2.1 DriverManager

The `DriverManager` defines the following methods:

```
public static Connection getConnection(
        String url, Properties info) throws SQLException;

public static Connection getConnection(
        String url, String user, String password) throws SQLException;

public static Connection getConnection(
        String url) throws SQLException;
```

Every time the `getConnection()` method is called, the `DriverManager` requests a new *physical* connection from the underlying `Driver`.

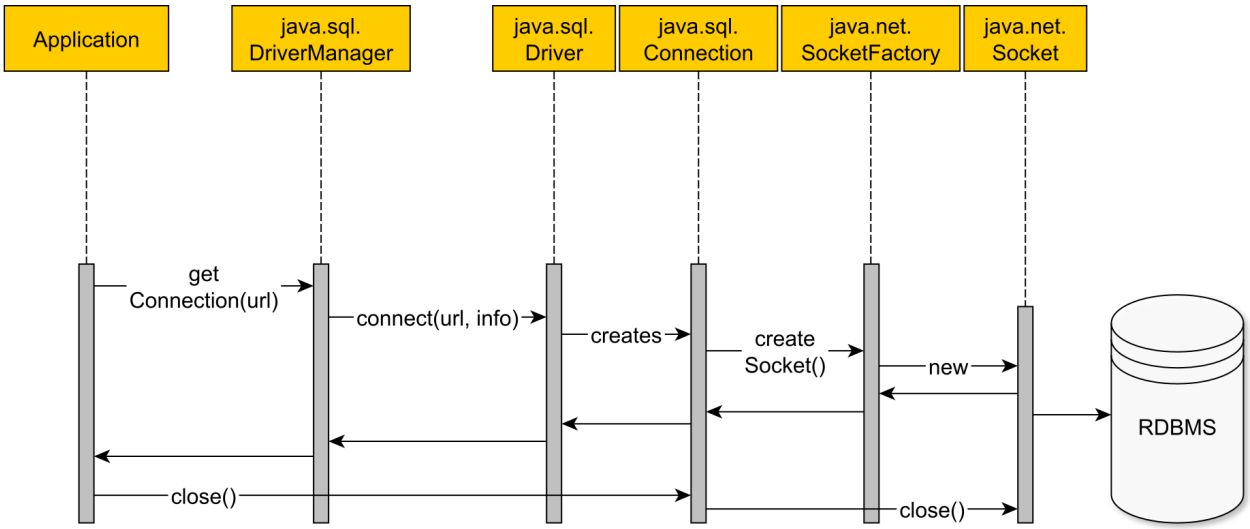

Figure 2.2: DriverManager connection

The first version of JDBC was launched in 1997, and it only supported the `DriverManager` utility for fetching database connections. Back then, Java was offering support for desktop applications which were often employing a two-tier architecture:

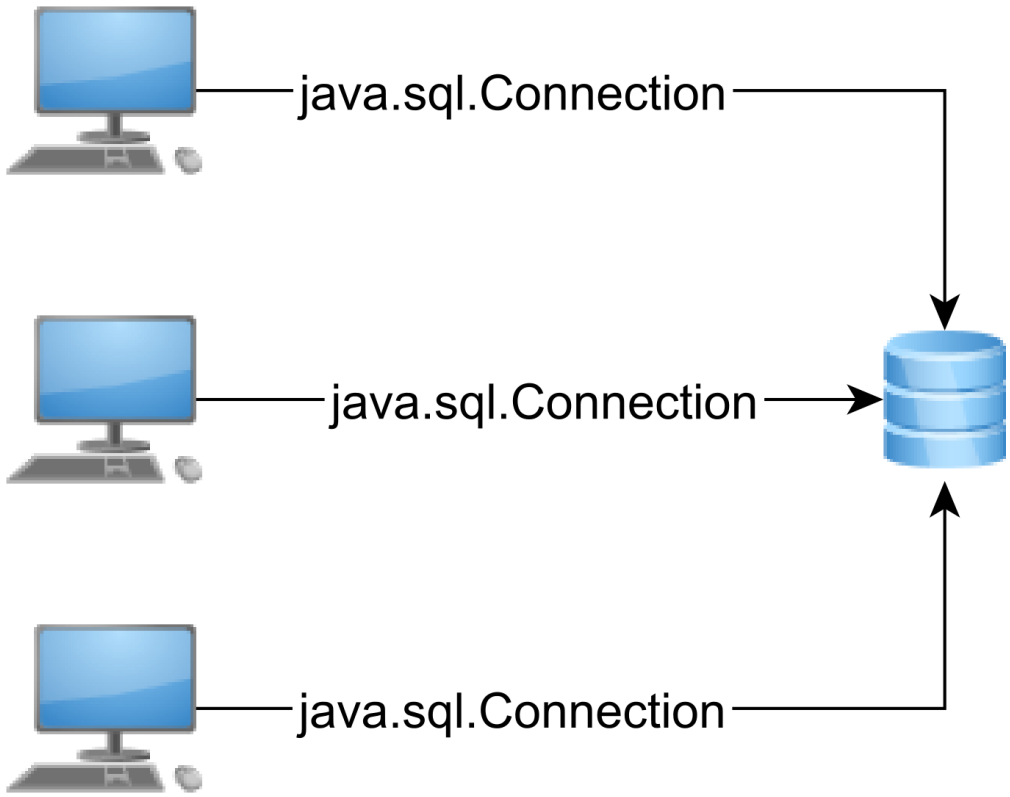

Figure 2.3: Two-tier architecture

In a two-tier architecture, the application is run by a single user, and each instance uses a dedicated database connection. The more users, the more database connections are required, and based on the underlying resources (hardware, operating system or licensing restrictions), each database server can offer a limited number of connections.

Oracle mainframe legacy

Oracle had gained its popularity in the era of mainframe computers when each client got a dedicated database connection.

Oracle assigns a distinct *schema* for each individual *user*, as opposed to other database systems where a schema is shared by multiple user accounts.

In PL/SQL, the *Packaged public variables* scope is bound to a *session*, instead of to the currently running transaction. The application developer must be extra cautious to unbind these variables properly since connections are often reused and old values might leak into newer transactions.

2.2 DataSource

In 1999, J2EE was launched along with JDBC 2.0 and an initial draft of JTA (Java Transaction API)[1], marking the beginning of Enterprise Java. Enterprise applications use a three-tier architecture, where the middle tier acts as a bridge between user requests and various data sources (e.g. relational databases, messaging queues).

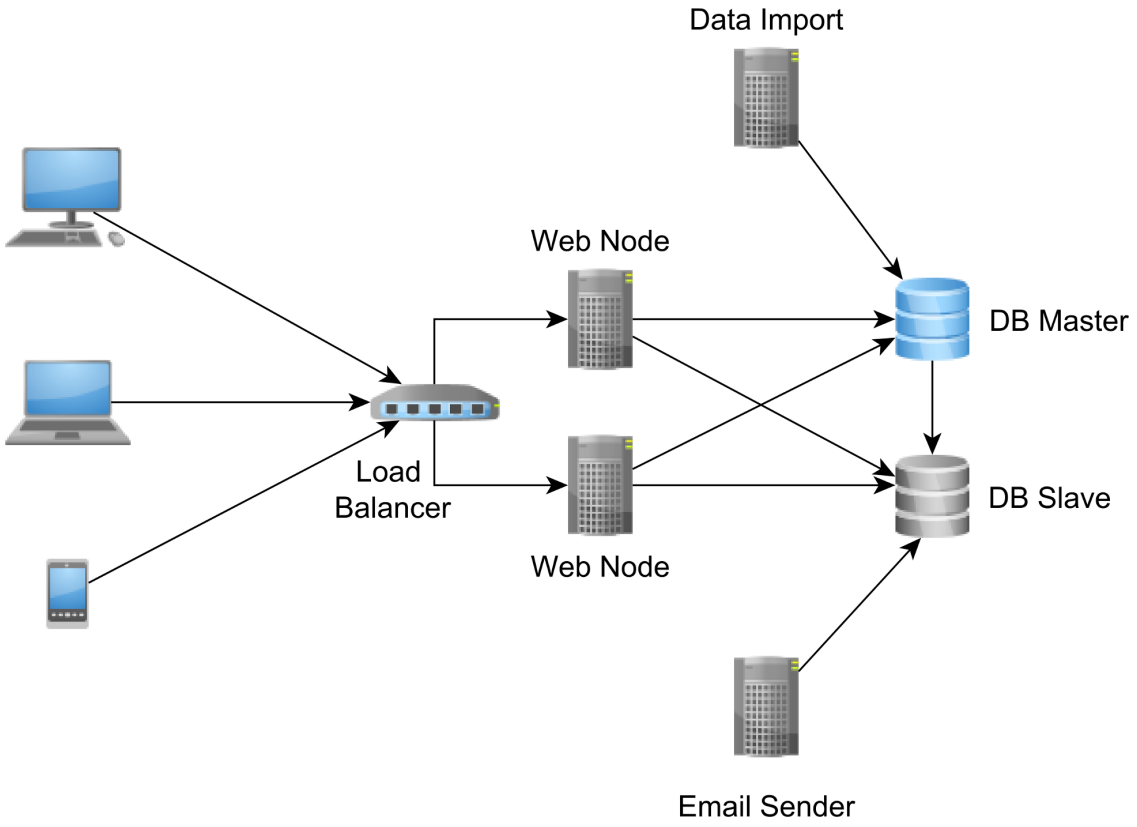

Figure 2.4: Three-tier architecture

Having an intermediate layer between the client and the database server has numerous advantages.

In a typical enterprise application, the user request throughput is greater than the available database connection capacity. As long as the connection acquisition time is tolerable (from the end-user perspective), the user request can wait for a database connection to become available. The middle layer acts as a database connection buffer that can mitigate user request traffic spikes by increasing request response time, without depleting database connections or discarding incoming traffic.

Because the intermediate layer manages database connections, the application server can also monitor connection usage and provide statistics to the operations team.

For this reason, instead of serving physical database connections, the application server

[1] https://jcp.org/en/jsr/detail?id=907

provides only logical connections (proxies or handles), so it can intercept and register how the client API interacts with the connection object.

A three-tier architecture can accommodate multiple data sources or messaging queue implementations. To span a single transaction over multiple sources of data, a distributed transaction manager becomes mandatory. In a JTA environment, the transaction manager must be aware of all logical connections the client has acquired as it has to commit or roll them back according to the global transaction outcome. By providing logical connections, the application server can decorate the database connection handles with JTA transaction semantics.

If the `DriverManager` is a physical connection factory, the `javax.sql.DataSource` interface is a logical connection provider:

```
Connection getConnection() throws SQLException;
Connection getConnection(String username, String password) throws SQLException;
```

The simplest `javax.sql.DataSource` implementation could delegate connection acquisition requests to the underlying `DriverManager`, and the connection request workflow would look like this:

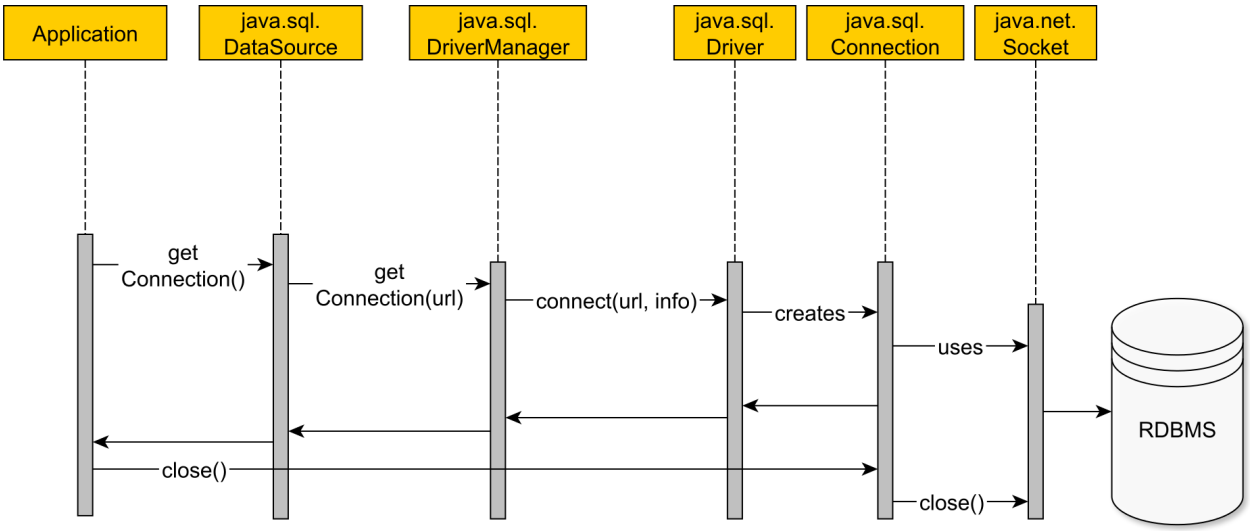

Figure 2.5: DataSource without connection pooling

1. The application data layer asks the `DataSource` for a database connection.
2. The `DataSource` uses the underlying driver to open a physical connection.
3. A physical connection is created, and a TCP socket is opened.
4. The `DataSource` under test does not wrap the physical connection, and it simply lends it to the application layer.
5. The application executes statements using the acquired database connection.
6. When the connection is no longer needed, the application closes the physical connection along with the underlying TCP socket.

Opening and closing database connections is a very expensive operation, so reusing them has the following advantages:

- It avoids both the database and the driver overhead for establishing a TCP connection.
- It prevents destroying the temporary memory buffers associated with each database connection.
- It reduces client-side JVM object garbage.

To visualize the cumulated overhead of establishing and closing database connections, the following test compares the total time it takes to open and close 1000 database connections of four different RDBMS against HikariCP[2] (one of the fastest stand-alone connection pooling solutions in the Java ecosystem).

Table 2.1: Database connection establishing overhead vs. connection pooling

Metric	Time (ms) DB_A	Time (ms) DB_B	Time (ms) DB_C	Time (ms) DB_D	Time (ms) HikariCP
min	11.174	5.441	24.468	0.860	0.001230
max	129.400	26.110	74.634	74.313	1.014051
mean	13.829	6.477	28.910	1.590	0.003458
p99	20.432	9.944	54.952	3.022	0.010263

When using a connection pooling solution, the connection acquisition time is between two and four orders of magnitude smaller. By reducing the connection acquisition interval, the overall transaction response time gets shorter too. All in all, in an enterprise application, reusing connections is a much better choice than always establishing them on a transaction basis.

Oracle XE connection handling limitation

While the Enterprise Edition does not entail any limitations, the Oracle 11g Express Edition throws the following exception when running very short transactions without using a connection pooling solution:

> ORA-12516, TNS:listener could not find available handler with matching protocol stack

A connection pooling solution can prevent these intermittent connection establishment failures and reduce the connection acquisition time as well.

[2] http://brettwooldridge.github.io/HikariCP/

2.2.1 Why is pooling so much faster?

To understand why the connection pooling solution performs so much better, it is important to figure out the connection pooling mechanism:

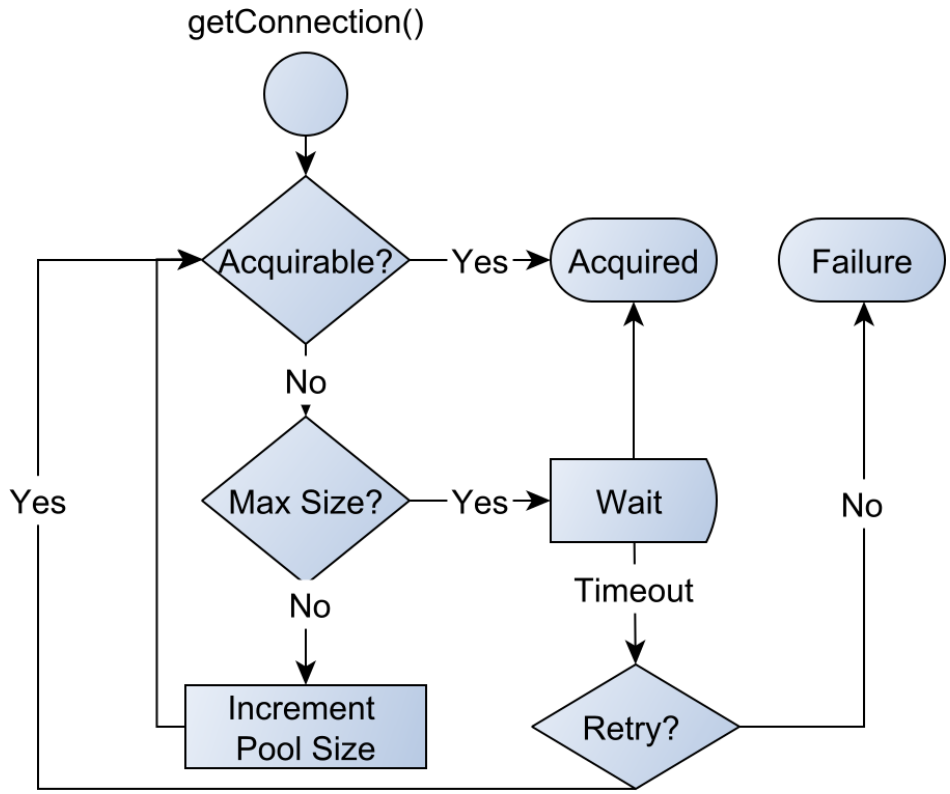

Figure 2.6: Connection acquisition request flow

1. When a connection is being requested, the pool looks for unallocated connections.
2. If the pool finds a free one, it will be handled to the client.
3. If there is no free connection, the pool will try to grow to its maximum allowed size.
4. If the pool already reached its maximum size, it will retry several times before giving up with a connection acquisition failure exception.
5. When the client closes the logical connection, the connection is released and returns to the pool without closing the underlying physical connection.

Most connection pooling solutions expose a `DataSource` implementation that either wraps an actual database-specific `DataSource` or the underlying `DriverManager` utility.

The logical connection lifecycle looks like this:

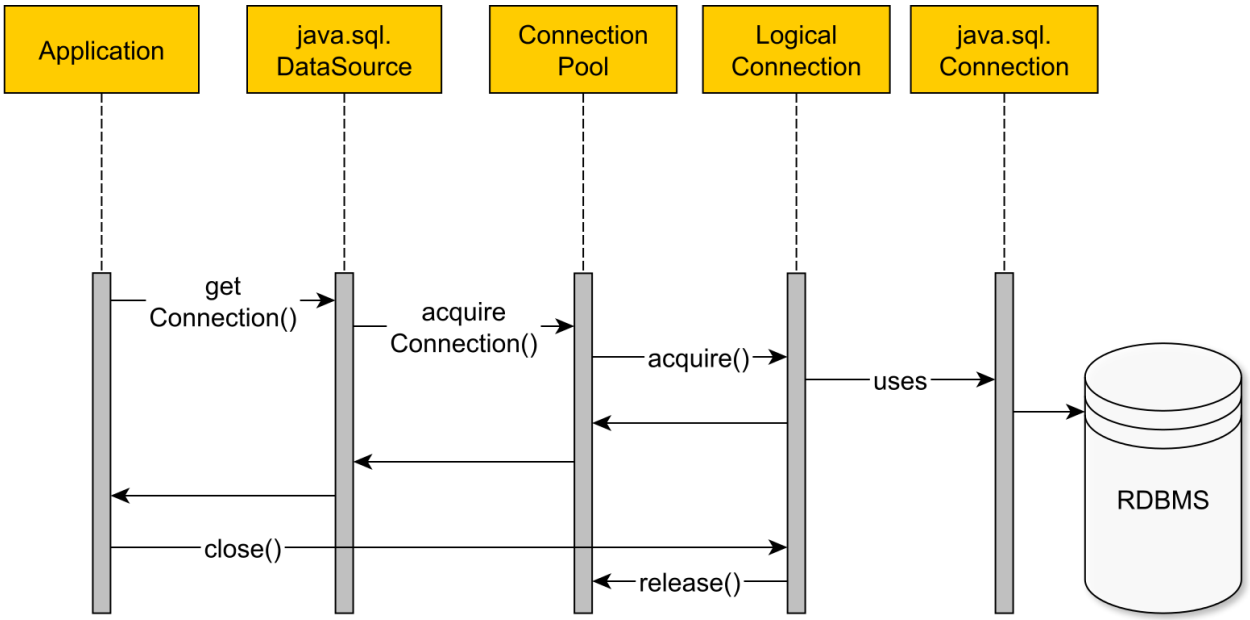

Figure 2.7: DataSource connection

The connection pool does not return the physical connection to the client, but instead it offers a proxy or a handle. When a connection is in use, the pool changes its state to *allocated* to prevent two concurrent threads from using the same database connection. The proxy intercepts the connection close method call, and it notifies the pool to change the connection state to *unallocated*.

Apart from reducing connection acquisition time, the pooling mechanism can also limit the number of connections an application can use at once.

The connection pool acts as a bounded buffer for the incoming connection requests. If there is a traffic spike, the connection pool will level it, instead of saturating all the available database resources.

All these benefits come at a price since configuring the right pool size is not a trivial thing to do. Provisioning the connection pool requires understanding the application-specific database access patterns and also connection usage monitoring.

Whenever the number of incoming requests surpasses the available request handlers, there are basically two options to avoid system overloading:

- discarding the overflowing traffic (affecting availability)
- queuing requests and wait for busy resources to become available (increasing response time).

Discarding the surplus traffic is usually a last resort measure, so most connection pooling solutions first attempt to enqueue overflowing incoming requests.

By putting an upper bound on the connection request wait time, the queue is prevented from growing indefinitely and saturating application server resources.

For a given incoming request rate, the relation between the queue size and the average enqueuing time is given by one of the most fundamental laws of queuing theory.

2.3 Queuing theory capacity planning

Little's Law[3] is a general-purpose equation applicable to any queueing system being in a stable state (the arrival rate is not greater than the departure rate).

According to Little's Law, the average time for a request to be serviced depends only on the long-term request arrival rate and the average number of requests in the system.

$$L = \lambda \times W$$

- L - average number of requests in the system (including both the requests being serviced and the ones waiting in the queue)
- λ - long-term average arrival rate
- W - average time a request spends in a system.

Assuming that an application-level transaction uses the same database connection throughout its whole lifecycle, and the average transaction response time is 100 milliseconds:

$$W = 100\,ms = 0.1\,s$$

If the average connection acquisition rate is 50 requests per second:

$$\lambda = 50\,\frac{connection\,requests}{s}$$

Then the average number of connection requests in the system will be:

$$L = \lambda \times W = 50 \times 0.1 = 5\,connection\,requests$$

A pool size of 5 can accommodate the average incoming traffic without having to enqueue any connection request. If the pool size is 3, then, on average, 2 requests will be enqueued and waiting for a connection to become available.

Little's Law operates with long-term averages, and that might not be suitable when taking into consideration intermittent traffic bursts. In a real-life scenario, the connection pool must adapt to short-term traffic spikes, and so it is important to consider the actual connection pool throughput.

[3] http://en.wikipedia.org/wiki/Little%27s_law

In queueing theory, throughput is represented by the departure rate (□), and, for a connection pool, it represents the number of connections offered in a given unit of time:

$$\mu = \frac{Ls}{Ws} = \frac{pool\ size}{connection\ lease\ time}$$

The following exercise demonstrates how queuing theory can help provisioning a connection pool to support various incoming traffic spikes.

Reusing the previous example configuration, the connection pool defines the following variables:

- There are at most 5 in-service requests (Ls), meaning that the pool can offer at most 5 connections.
- The average service time (Ws) or the connection lease time is 100 milliseconds.

As expected, the connection pool can deliver up to 50 connections per second.

$$\mu = \frac{Ls}{Ws} = 50\,\frac{connection\ requests}{s}$$

When the arrival rate equals departure rate, the system is saturated with all connections being in use.

$$\lambda = \mu = \frac{Ls}{Ws}$$

If the arrival rate outgrows the connection pool throughput, the overflowing requests must wait for connections to become available.

A one-second traffic burst of 150 requests is handled as follows:

- The first 50 requests can be served in the first second.
- The next 100 requests are first enqueued and processed in the following 2 seconds.

$$\mu = \frac{Ls}{Ws} = \frac{5}{0.1} = \frac{Lq}{Wq} = \frac{10}{0.2}$$

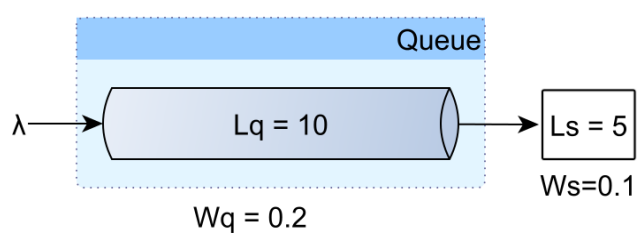

Figure 2.8: Little's Law queue

For a constant throughput, the number of enqueued connection requests (*Lq*) is proportional to the connection acquisition time (*Wq*).

The total number of requests in any given spike is calculated as follows:

$$Lspike = \lambda spike \times Wspike$$

The total time required to process the spike is given by the following formula:

$$W = \frac{Lspike}{\mu} = \frac{\lambda spike \times Wspike}{\lambda}$$

The number of enqueued connection requests and the time it takes to process them is expressed by the following equations:

$$Lq = Lspike - Ls$$

$$Wq = W - 1$$

Assuming there is a traffic spike of 250 requests per second, lasting for 3 seconds.

$$\lambda spike = 250\,\frac{requests}{s}$$

$$Wspike = 3\,s$$

The 750 requests spike takes 15 seconds to be fully processed.

$$Lspike = 250\,\frac{requests}{s} \times 3\,s = 750\,requests$$

$$W = \frac{750\,requests}{50\,\frac{requests}{s}} = 15\,s$$

The queue size grows to 700 entries, and it requires 14 seconds for all connection requests to be serviced.

$$Lq = Lspike - Ls = 700\,requests$$

$$Wq = W - 1 = 14\,s$$

2.4 Practical database connection provisioning

Even if queuing theory provides insight into the connection pool behavior, the dynamics of enterprise systems are much more difficult to express with general-purpose equations, and metrics become fundamental for resource provisioning. By continuously monitoring the connection usage patterns, it is much easier to react and adjust the pool size when the initial configuration does not hold anymore.

Unfortunately, many connection pooling solutions only offer limited support for monitoring and failover strategies, and that was the main reason for building FlexyPool[4]. Supporting the most common connection pooling frameworks, this open source project offers the following connection usage metrics:

Table 2.2: FlexyPool metrics

Name	Description
concurrent connection requests	How many connections are being requested at once
concurrent connections	How many connections are being used at once
maximum pool size	If the target `DataSource` uses adaptive pool sizing, this metric will show how the pool size varies with time
connection acquisition time	The time it takes to acquire a connection from the target `DataSource`
overall connection acquisition time	The total connection acquisition interval (including retries)
retry attempts	The connection acquisition retry attempts
overflow pool size	How much the pool size can grow over the maximum size until timing out the connection acquisition request
connection lease time	The duration between the moment a connection is acquired and the time it gets released

While metrics are important for visualizing connection usage trends, in case of an unforeseen traffic spike, the connection acquisition time could reach the `DataSource` timeout threshold.

The failover mechanism applies various strategies to prevent timed-out connection requests from being discarded. While a batch processor can retry a failing request (although it increases transaction response time), in a web application, the user is much more sensitive to unavailability or long-running transactions.

[4] https://github.com/vladmihalcea/flexy-pool